

Inverse Compositional Algorithm Project

Project Overview

This project is a Python implementation of the Inverse Compositional Algorithm for parametric image registration, based on the academic publication *The Inverse Compositional Algorithm for Parametric Registration* published in *Image Processing On Line* in 2016. The work started as a translation and adaptation of the original C++ reference implementation into a lightweight, portable, and easier-to-use Python codebase. It was later aligned with the improved 2018 version of the method, which introduced better convergence behavior and lower computational cost.

The objective of the project was not only to reproduce the original algorithm, but also to make it practical for experimentation, validation, and future acceleration. The result is a modular computer vision project focused on estimating geometric transformations between images through iterative optimization. It is designed for research, benchmarking, and engineering use cases where image alignment accuracy and reproducibility matter.

Technical Scope

At its core, the project implements several variants of the inverse compositional framework for image alignment. The standard quadratic formulation is available, as well as a robust version that can better handle noise, outliers, and partial image inconsistencies. A pyramidal coarse-to-fine strategy is also included to improve convergence when the displacement between images is too large for a single-scale optimization process.

The implementation is structured into clear processing modules. Core mathematical operations such as Jacobian and Hessian evaluation are separated from image warping, bicubic interpolation, parameter updates, and optimization utilities. This modular design makes the code easier to test, maintain, and extend.

The transformation framework supports multiple motion models, including translation, Euclidean motion, similarity, affinity, and homography parameterizations. In practice, the validation work focused especially on translation, rotation, Euclidean, and zoom-like transformations, using controlled examples to compare estimated motion parameters against known ground truth values.

Engineering Work Delivered

A large part of the work consisted of turning a research algorithm into an organized and reusable software project. The codebase includes:

- a configurable Python implementation based on NumPy, SciPy, and scikit-image,
- robust error formulations such as quadratic, truncated quadratic, German-McClure, Lorentzian, and Charbonnier penalties,
- a pyramidal multi-scale registration pipeline,
- configuration handling through .ini files,
- Jupyter notebooks for experiments, debugging, demonstrations, and reproducibility,
- unit tests covering mathematical building blocks such as Jacobian, Hessian, and numerical validity checks. The project also includes a Google Colab notebook to make the implementation easier to run in

a cloud environment. This is useful for sharing experiments, demonstrating results, or allowing non-technical users to execute the workflow without having to reproduce the full local setup.

Data Preparation and Validation

To validate the implementation, the project uses image data inspired by established computer vision benchmarks. Raw images were sourced from the Middlebury dataset, a well-known reference in optical flow and image registration research. On top of these source images, controlled geometric transformations were generated to create processed test samples with known target parameters.

A dedicated validation methodology was defined for the project. The approach was to start from simple cases, validate results against the original C++ implementation, and progressively test more difficult configurations such as robust error functions and multi-scale processing. The notebooks show how synthetic transformed images were created and how the estimated parameters were compared with the expected ground truth.

This validation workflow is important because image registration algorithms are highly sensitive to implementation details such as interpolation, gradient computation, boundary handling, and parameter update rules. By documenting test cases and comparing outputs with a trusted reference implementation, the project demonstrates both technical rigor and reproducibility.

Performance and Acceleration

Beyond the initial NumPy version, the project was extended with a hardware-accelerated implementation based on TensorFlow and Keras. This accelerated branch re-expresses the inverse compositional optimization pipeline in tensor operations and introduces custom layers for batch-oriented processing. This version provides an average speedup of about 10x compared with the NumPy implementation, with even larger gains possible when batches share the same source image and convergence settings.

This part of the work shows a progression from research reproduction to performance engineering. It demonstrates the ability to take a mathematically complex algorithm and redesign it for modern accelerated execution environments.

Outcome

Overall, this project showcases applied work at the intersection of computer vision, numerical optimization, scientific Python, and performance-oriented engineering. It combines algorithm implementation, code modularization, reproducible experimentation, dataset preparation, testing, and acceleration into one coherent deliverable.

For a portfolio, this project highlights several strengths: the ability to translate academic methods into usable software, build clean experimental workflows, validate results carefully, and push an implementation toward faster execution on modern frameworks. It is a strong example of end-to-end technical ownership on a non-trivial image processing problem.